

Introduction to High-Level Synthesis — Part I: Theory

- Introduction to High-Level Synthesis
 - หลักสูตรอบรมระดับเริ่มต้นถึงระดับกลาง (Beginner → Intermediate)
 - สารบัญ
- Part I — ภาคทฤษฎี
 - บทที่ 1 — High-Level Synthesis คืออะไร?
 - บทที่ 2 — กระบวนการทำงานของ HLS
 - บทที่ 3 — กลไกภายในของ HLS
 - บทที่ 4 — ภาษาต้นทางและ Coding Style
 - บทที่ 5 — Directives / Pragmas ที่ใช้บ่อย
 - บทที่ 6 — Interface Synthesis

Introduction to High-Level Synthesis

หลักสูตรอบรมระดับเริ่มต้นถึงระดับกลาง (Beginner → Intermediate)

กลุ่มเป้าหมาย: นักศึกษาวิศวกรรมชั้นปีที่ 3–4 และวิศวกรจบใหม่ที่มีพื้นฐาน RTL (Verilog/SystemVerilog) และเคยใช้งาน Open-source EDA (Yosys / OpenROAD / Icarus / COCOTB)

ระยะเวลา: 3 วัน (ทฤษฎี 1 วัน + ปฏิบัติ 2 วัน) — ปรับได้ตามความเหมาะสม

เครื่องมือหลัก: Bambu HLS (open-source), Yosys, OpenROAD, Icarus Verilog, COCOTB

PDK อ้างอิง: IHP SG13G2 (130 nm SiGe BiCMOS) และ SKY130

สารบัญ

Part I — ภาคทฤษฎี

- บทที่ 1 — High-Level Synthesis คืออะไร?
- บทที่ 2 — กระบวนการทำงานของ HLS
- บทที่ 3 — กลไกภายในของ HLS: Scheduling, Binding, Allocation
- บทที่ 4 — ภาษาต้นทางและ Coding Style
- บทที่ 5 — Directives / Pragmas ที่ใช้บ่อย
- บทที่ 6 — Interface Synthesis

Part II — ภาคปฏิบัติ (อยู่ในไฟล์ `hls-course-part2-practice.md`)

- การติดตั้ง Bambu HLS
- Lab 1 — Simple Adder
- Lab 2 — FIR Filter with Pipelining

10. Lab 3 — Matrix Multiplication

11. Lab 4 — Dataflow Streaming

12. การตรวจสอบ (Verification)

13. การนำไปใช้กับ OpenROAD (IHP SG13G2 / SKY130)

Part I — ภาคทฤษฎี

บทที่ 1 — High-Level Synthesis คืออะไร?

1.1 นิยาม

High-Level Synthesis (HLS) คือ กระบวนการแปลงภาษาระดับสูง (C / C++ / SystemC / MATLAB) ไปเป็น Register-Transfer Level (RTL) ที่อธิบายด้วย HDL (Verilog / VHDL) โดยอัตโนมัติ ซึ่งต่างจากการเขียน RTL ด้วยมือ (manual RTL design) ที่นักออกแบบต้องระบุทุก clock cycle, ทุก state, ทุก register ด้วยตนเอง

สรุปสั้น ๆ: *HLS* = “อธิบายว่าทำอะไร (What) → ให้เครื่องมือคิดว่าทำอย่างไร (How)”

1.2 เปรียบเทียบ: RTL แบบดั้งเดิม vs HLS

มิติ	RTL Design (Manual)	High-Level Synthesis
ภาษา	Verilog / VHDL / SystemVerilog	C / C++ / SystemC
ระดับนามธรรม	Cycle-accurate	Algorithmic / behavioral
ความเร็วในการออกแบบ	ช้า (วัน-สัปดาห์ต่อบล็อก)	เร็ว (ชั่วโมง)
ควบคุม micro-architecture	เต็มที่	ผ่าน directives/pragmas
ความยากของ design space exploration	ต้องเขียน RTL หลายเวอร์ชัน	เปลี่ยน directive แล้วรันใหม่
ผลลัพธ์เชิงคุณภาพ (QoR)	ขึ้นกับความเชี่ยวชาญ	ดีใกล้เคียงกันเมื่อปรับ directive เหมาะสม
การ verify	ต้องเขียน testbench HDL	ใช้ C testbench เดิมได้เลย (C/RTL co-sim)

1.3 ประวัติศาสตร์ (ย่อ)

- 1980s — งานวิจัย HLS เริ่มต้น (Synopsys Behavioral Compiler, Intel/Synopsys/Academic prototypes)
- 1990s — การใช้งานจริงในอุตสาหกรรม DSP
- 2000s — SystemC กลายเป็นมาตรฐาน IEEE 1666
- 2010s — Xilinx Vivado HLS / Vitis HLS ทำให้ HLS เป็น mainstream สำหรับ FPGA
- 2020s — Open-source HLS (Bambu, Dynamic) + การผสานกับ OpenROAD ทำให้ ASIC design เข้าถึงได้

1.4 เครื่องมือ HLS ที่ใช้ในอุตสาหกรรมและงานวิจัย

เครื่องมือ	ผู้พัฒนา	License	ใช้กับ
Vitis HLS	AMD/Xilinx	Commercial (free Webpack)	FPGA
Intel HLS Compiler	Intel	Commercial (free edition)	FPGA
Catapult HLS	Siemens EDA (Mentor)	Commercial	ASIC / FPGA
Stratus HLS	Cadence	Commercial	ASIC / FPGA
Bambu	Politecnico di Milano	Open-source (GPL)	ASIC / FPGA / OpenROAD
Dynamic	EPFL	Open-source (Apache 2.0)	ASIC (research)
LegUp	Univ. of Toronto	Open-source (legacy)	FPGA (research)
HLS4ML	CERN/CMU	Open-source (BSD)	ML on FPGA

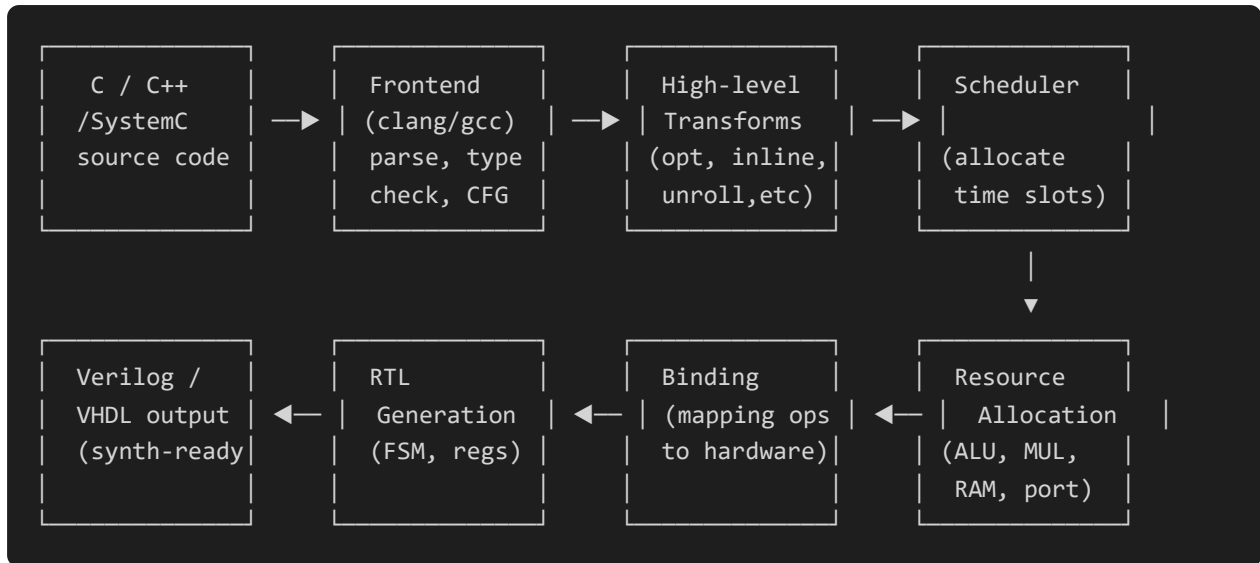
สำหรับหลักสูตรนี้ เราจะใช้ **Bambu HLS** เนื่องจากเป็น open-source และเชื่อมต่อกับ Yosys + OpenROAD ได้โดยตรง (เหมาะกับ PDK IHP SG13G2 / SKY130) แต่จะอธิบายแนวคิดในเชิง tool-agnostic เพื่อให้นำไปใช้กับเครื่องมือ commercial ได้ด้วย

1.5 ทำไมต้องเรียน HLS?

1. **Productivity** — เขียนน้อยลง 10x เมื่อเทียบกับ manual RTL
2. **Design Space Exploration (DSE)** — ลองหลาย micro-architecture ได้เร็ว
3. **Reuse** — testbench เดียวกัน verify ทั้ง C model และ RTL
4. **Algorithm Focus** — วิศวกรมุ่งเน้น “ทำอะไร” ไม่ใช่ “ทำอย่างไร”
5. **Open-source Ecosystem** — สอดคล้องกับ flow Yosys + OpenROAD ที่ใช้อยู่แล้ว

บทที่ 2 — กระบวนการทำงานของ HLS

2.1 Flow ภาพรวม



2.2 Frontend (ส่วนหน้า)

ใช้ compiler infrastructure (clang, gcc, **혹은** SystemC simulator) เพื่อ:

- Lexical/Syntax Analysis — แยก token, parse เป็น AST
- Type Checking — ตรวจสอบชนิดข้อมูล
- Control Flow Graph (CFG) — แทน execution path ของโปรแกรม
- Data Flow Analysis — ติดตามการใช้ตัวแปร (def-use chain)

ผลลัพธ์: Intermediate Representation (IR) เช่น LLVM IR หรือ IR เฉพาะของเครื่องมือ

2.3 High-Level Transformations

ก่อน scheduling เครื่องมืออาจทำ optimization:

- Function Inlining — แทน call ด้วย body เพื่อให้ scheduler เห็นภาพรวม
- Loop Transformations — unroll, fusion, fission, interchange, tiling
- Dead Code Elimination — ลบโค้ดที่ไม่ได้ใช้
- Constant Propagation — แทนค่าคงที่ตอนคอมไพล์

2.4 Scheduling — จัดสรร “เวลา”

Scheduling ตัดสินใจว่าแต่ละ operation จะทำงานใน clock cycle ใด โดย:

- เคารพ data dependencies (ไม่ให้ใช้ค่าก่อนผลิต)

- เคารพ **resource constraints** (ไม่เกินจำนวน functional unit ที่มี)
- พยายามบรรลุ **target latency / throughput**

อัลกอริทึมที่นิยม: - **ASAP** (As Soon As Possible) - **ALAP** (As Late As Possible) - **List Scheduling** (priority-based, เร็วและนิยมที่สุด) - **Force-Directed Scheduling**

2.5 Allocation & Binding — จัดสรร “ทรัพยากร”

- **Allocation** — เลือกว่าจะมี functional unit อะไรบ้าง เช่น 2x adder, 1x multiplier
- **Binding** — map operation แต่ละตัวไปยัง functional unit ตัวใด
- **Register Allocation** — เลือกว่าตัวแปรใดอยู่ใน register ตัวใด รวมถึง register sharing

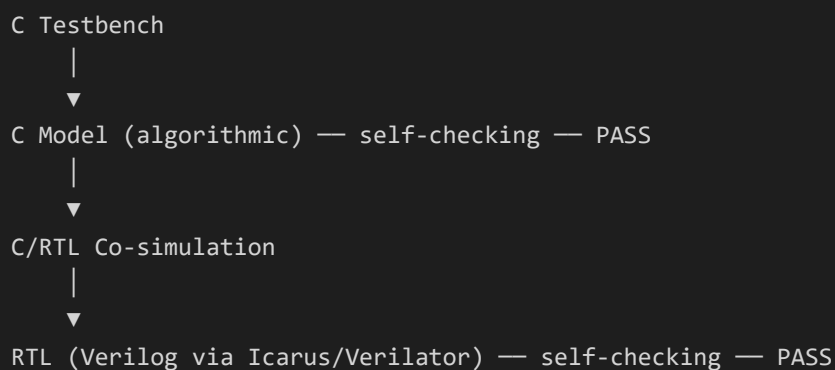
2.6 RTL Generation

สร้างไฟล์ Verilog/VHDL:

- **FSM** จาก CFG (state register, next-state logic)
- **Datapath** (functional units, muxes, wires)
- **Control Unit** (FSM + interface protocol)
- **Memory** (registers, BRAM inference, FIFO)
- **Interface** (ap_start, ap_done, AXI, FIFO, RAM ports)

2.7 Verification Flow (สำคัญมาก)

HLS มี co-simulation เป็นจุดแข็ง:

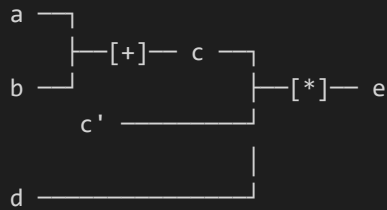


ข้อดี: ใช้ testbench ตัวเดียว ไม่ต้องเขียน HDL testbench ใหม่

บทที่ 3 — กลไกภายในของ HLS

3.1 Control-Data Flow Graph (CDFG)

HLS แทนโปรแกรมด้วย CDFG: - **Control Flow** — basic blocks + branches - **Data Flow** — operations + data dependencies



3.2 ตัวอย่าง: Scheduling แบบง่าย

โค้ด:

```
c = a + b;  
e = c * d;  
f = e + 1;
```

สมมติ adder ใช้ 1 cycle, multiplier ใช้ 2 cycles:

Cycle	0	1	2	3
Op1 +	✓			
Op2 *		✓	✓	
Op3 +				✓

ผลลัพธ์: Latency = 4 cycles, 1 adder, 1 multiplier

ถ้าเราเพิ่ม multiplier อีก 1 ตัว (resource constraint เปลี่ยน) → schedule จะสั้นลงได้

3.3 Resource Allocation & Binding

ตัวอย่างการ binding:

```
Op1: a + b    → adder_0  
Op3: e + 1    → adder_0    (register share กับ Op1)  
Op2: c * d    → mult_0
```

Cost tradeoffs: - Sharing functional unit → ใช้พื้นที่น้อย แต่ latency เพิ่ม - ไม่ share → เร็วขึ้น แต่พื้นที่เพิ่ม

3.4 Register Allocation

ตัวแปรที่ “มีชีวิต” (live) พร้อมกันต้องอยู่คนละ register:

```
int a, b, c, d, e;
c = a + b;    // a,b,c live
e = c + d;    // a,b dead; c,d,e live
```

Solution: graph coloring algorithm (เช่น Chaitin’s algorithm)

3.5 Memory Architecture

HLS ต้องตัดสินใจว่า array จะเก็บที่ไหน: - **Register** — เร็วสุด แต่ใช้ FF จำนวนมาก - **Distributed RAM (LUTRAM)** — เร็ว, จำกัดขนาด - **Block RAM (BRAM)** — ใหญ่, 1–2 cycle latency - **External memory** — ช้า, ใช้ bus protocol

ตัวเลือกนี้ถูกควบคุมด้วย **directive** (ดูบทที่ 5)

บทที่ 4 — ภาษาต้นทางและ Coding Style

4.1 ภาษาที่รองรับ

ภาษา	Bambu	Vitis HLS	Catapult
C	✓	✓	✓
C++	✓	✓	✓
SystemC	✓ (จำกัด)	ผ่าน wrapper	✓
OpenCL	—	✓	✓
MATLAB/Simulink	—	ผ่าน HDL Coder	✓

4.2 Arbitrary Precision Data Types

ภาษา C ใช้ int (32-bit), double (64-bit) ซึ่งไม่ตรงกับ hardware จริงเสมอ HLS จึงมี type พิเศษ:

Xilinx (Vitis HLS):

```
#include "ap_int.h"
ap_int<8> x;    // signed 8-bit
ap_uint<16> y;  // unsigned 16-bit
ap_fixed<16,8> fx; // Q8.8 fixed-point
```


Bambu (ac_types — Mentor/Siemens):

```
#include "ac_int.h"
ac_int<8,false> x;    // unsigned 8-bit
ac_int<8,true> y;     // signed 8-bit
ac_fixed<16,8,true,AC_RND,AC_SAT> fx; // fixed-point
```

เคล็ดลับ: ใช้ type เล็กที่สุดเท่าที่จำเป็น → ลด area, เร็วขึ้น

4.3 hls::stream (FIFO Channel)

สำหรับ streaming interface:

```
#include "hls_stream.h"
void producer(hls::stream<int>& out) {
    out.write(42);
}
void consumer(hls::stream<int>& in) {
    int x = in.read();
}
```

4.4 Coding Style สำหรับ HLS

หลักการสำคัญ:

- หลีกเลี่ยง dynamic memory (malloc, free, new) — HLS ต้องการ static memory
- หลีกเลี่ยง recursion — HLS ไม่ synthesize ได้ (Bambu/Vitis จะ error)
- หลีกเลี่ยง system calls (printf, scanf) — ใช้เฉพาะใน testbench
- ใช้ const — ช่วยให้ constant propagation ทำงาน
- Loop bounds ต้องรู้ตอน compile-time (อย่างน้อยสำหรับ unroll)
- หลีกเลี่ยง pointer arithmetic ที่ซับซ้อน — HLS วิเคราะห์ไม่ได้

ตัวอย่าง: Bad vs Good

```
// ❌ Bad: dynamic bound
for (int i = 0; i < n; i++) { ... } // n มาจาก input → ต้องระบุ max

// ✅ Good: known bound (หรือใช้ #pragma)
for (int i = 0; i < 16; i++) { ... }
```

```
// ❌ Bad: dynamic memory
int *buf = malloc(N * sizeof(int));

// ✅ Good: static array
int buf[1024];
```

```
// ❌ Bad: pointer alias
void foo(int *a, int *b) { *a = *b + 1; }
// HLS อาจ assume a และ b ซ้ำที่เดียวกัน (alias) → schedule ไม่กล้า parallel

// ✅ Good: ใช้ restrict
void foo(int * restrict a, int * restrict b) { *a = *b + 1; }
```

4.5 Function Hierarchy & Top Function

HLS เลือก top function เป็นจุดเริ่มต้น synthesis:

```
// top function – argument จะกลายเป็น port
void dut(int a, int b, int& c) {
  #pragma HLS PIPELINE
  c = a + b;
}

int main() { // ไม่ synthesize
  int a=1, b=2, c;
  dut(a, b, c);
  return (c == 3) ? 0 : 1;
}
```

Vitis HLS ใช้ `#pragma HLS INTERFACE` ระบุ port protocol Bambu ใช้ `--top-function` flag บน command line

บทที่ 5 — Directives / Pragmas ที่ใช้บ่อย

คำสั่งเหล่านี้บอก HLS ว่า “อยากให้ micro-architecture เป็นอย่างไร” ใน Vitis HLS ใช้ `#pragma HLS ...` ใน Bambu ใช้ `#pragma HLS ...` เช่นกัน หรือระบุผ่าน flag `--pragma=...`

5.1 PIPELINE

วัตถุประสงค์: เร่ง throughput — ทำให้ loop ใหม่เริ่มได้ทุก ๆ N cycles (II = Initiation Interval)

```
void foo(int a[16], int b[16], int c[16]) {
#pragma HLS PIPELINE II=1
    for (int i = 0; i < 16; i++) {
        c[i] = a[i] + b[i];
    }
}
```

ผลลัพธ์: - ก่อน PIPELINE: 16 iterations × (latency) = ~16+ cycles - หลัง PIPELINE II=1: 1 iteration / clock → 16 cycles + pipeline fill/drain

ค่า II: - II=1 คือเป้าหมายสูงสุด (เร็วสุด) - ถ้า II ต่ำกว่าที่เป็นไปได้ HLS จะบอก warning

5.2 UNROLL

วัตถุประสงค์: สร้าง hardware ซ้ำหลายชุดเพื่อทำงานขนาน

```
#pragma HLS UNROLL factor=4
for (int i = 0; i < 16; i++) {
    c[i] = a[i] * b[i];
}
```

- **UNROLL** (ไม่มี factor) = unroll เต็ม
- **UNROLL factor=4** = unroll 4 ครั้ง (4x parallel units)
- Cost: พื้นที่เพิ่ม, Benefit: throughput เพิ่ม

5.3 ARRAY_PARTITION

วัตถุประสงค์: แบ่ง array เพื่อให้เข้าถึงหลาย element พร้อมกันได้

```
int buf[16];
#pragma HLS ARRAY_PARTITION variable=buf complete
// ตอนนี้ buf[0]..buf[15] อยู่คนละ register/RAM → เข้าถึงพร้อมกันได้
```

Modes: - **complete** — แยกทุก element - **block factor=N** — แบ่งเป็น block ละ N - **cyclic factor=N** — สลับแบบ round-robin

5.4 DATAFLOW

วัตถุประสงค์: ทำให้ function หลาย ๆ ตัวทำงานพร้อมกัน (task-level parallelism)

```
void kernel(in_t a, in_t b, out_t& c) {
#pragma HLS DATAFLOW
    stage1(a, b, t1);
    stage2(t1, t2);
    stage3(t2, c);
}
```

ผลลัพธ์: pipeline ระหว่าง function ใช้ hls::stream เชื่อมต่อ

5.5 INLINE

วัตถุประสงค์: เอา body ของ function มาแทนที่ call site

```
#pragma HLS INLINE
int square(int x) { return x*x; }
```

เมื่อไหร่ใช้: เมื่อ function เล็กและต้องการ share resource กับ caller

5.6 LOOP_FLATTEN / LOOP_MERGE

```
#pragma HLS LOOP_FLATTEN
for (int i = 0; i < 4; i++)
    for (int j = 0; j < 4; j++)
        ... ;
// → กลายเป็น Loop เดียว 16 iterations
```

Benefit: เพิ่ม optimization opportunity

5.7 DEPENDENCE

บอก HLS ว่าไม่มี dependency (เมื่อ HLS อนุมานว่ามี):

```
#pragma HLS DEPENDENCE variable=a inter false
// ไม่มี dependency ระหว่าง iteration ของ array a
```

5.8 สรุป Directives

Directive	ผลต่อ Area	ผลต่อ Latency	ใช้บ่อยที่ไหน
PIPELINE	+	-	inner loop
UNROLL	++	-	loop เล็ก
ARRAY_PARTITION	++	-	memory bottleneck
DATAFLOW	+	-	pipeline function
INLINE	-	+	small helper
LOOP_FLATTEN	+/-	-	nested loop
DEPENDENCE	0	-	false dep

บทที่ 6 — Interface Synthesis

6.1 Block-Level Interface (Default)

HLS สร้าง port protocol เริ่มต้น:

Port	ความหมาย
ap_start	เริ่มทำงาน (input, pulse)
ap_done	ทำงานเสร็จ (output, pulse)
ap_idle	ยังไม่เริ่ม/ทำเสร็จแล้ว (output, level)
ap_ready	พร้อมรับ input ใหม่ (output, level)

6.2 Function Argument → Port

```
void dut(int a, int b, int& c);  
// → ports: a (input), b (input), c (output)
```

โดย default ใช้ handshake protocol: - a_in (data), a_vld (valid), a_rd (ready) — หรือ variant อื่น

6.3 AXI Interface

Vitis HLS / Bambu สามารถสร้าง AXI4, AXI4-Lite, AXI4-Stream:

```
#pragma HLS INTERFACE m_axi port=a bundle=A depth=1024
#pragma HLS INTERFACE s_axilite port=return bundle=CTRL
```

AXI4-Stream (สำหรับ streaming data):

```
#pragma HLS INTERFACE axis port=in_stream
#pragma HLS INTERFACE axis port=out_stream
```

6.4 FIFO / Stream Interface

```
#include "hls_stream.h"
void dut(hls::stream<int>& in, hls::stream<int>& out) {
#pragma HLS INTERFACE axis port=in
#pragma HLS INTERFACE axis port=out
    out.write(in.read() + 1);
}
```

6.5 RAM Interface

```
#pragma HLS INTERFACE bram port=ram
```

HLS จะ infer BRAM (block RAM) โดยอัตโนมัติ — ใช้สำหรับ array ที่ใหญ่

6.6 Memory Mapped vs Streaming

	Memory-Mapped (AXI)	Streaming (FIFO)
Address	มี	ไม่มี
Latency	สูง (bus)	ต่ำ
Throughput	จำกัดด้วย bus	ต่อ clock
ใช้เมื่อ	CPU เข้าถึง	data pipeline

ดู Part II สำหรับการปฏิบัติกับ Bambu HLS, การ verify, และ integration กับ OpenROAD

Introduction to High-Level Synthesis — Part II: Practice

- Introduction to High-Level Synthesis — Part II (ภาคปฏิบัติ)
 - สารบัญ Part II
 - บทที่ 7 — การติดตั้ง Bambu HLS
 - บทที่ 8 — Lab 1: Simple Adder
 - บทที่ 9 — Lab 2: FIR Filter
 - บทที่ 10 — Lab 3: Matrix Multiplication
 - บทที่ 11 — Lab 4: Dataflow Streaming
 - บทที่ 12 — Verification
 - บทที่ 13 — Integration กับ OpenROAD
 - บทที่ 14 — Best Practices และ Pitfalls
 - ภาคผนวก — Cheat-Sheet
 - แบบฝึกหัดรวม (Final Project)

Introduction to High-Level Synthesis — Part II (ภาคปฏิบัติ)

ต่อจาก Part I (ทฤษฎี) — เอกสารฉบับนี้เน้นการลงมือทำจริงกับ **Bambu HLS** และผสานเข้ากับ flow Open-source (Yosys / OpenROAD / Icarus / COCOTB)

สารบัญ Part II

- 7. การติดตั้ง Bambu HLS
- 8. Lab 1 — Simple Adder (Hello World)
- 9. Lab 2 — FIR Filter with Pipelining
- 10. Lab 3 — Matrix Multiplication
- 11. Lab 4 — Dataflow Streaming
- 12. Verification ด้วย Icarus และ COCOTB
- 13. Integration กับ OpenROAD (IHP SG13G2 / SKY130)
- 14. Best Practices และ Pitfalls
- 15. ภาคผนวก: Cheat-Sheet

บทที่ 7 — การติดตั้ง Bambu HLS

7.1 ทำไมเลือก Bambu?

Bambu เป็น HLS tool open-source ที่:

- เขียนด้วย C++ (ดูแลโดย PoliMi)
- รับ C/C++ → ส่ง Verilog/VHDL

- ออกแบบมาเพื่อ ASIC flow (ไม่ใช่แค่ FPGA)
- ใช้ร่วมกับ Yosys สำหรับ logic synthesis
- รองรับ multiple backends: Vivado (Xilinx FPGA), OpenROAD (ASIC), Catapult, custom
- ไม่ผูกกับ vendor

7.2 ความต้องการของระบบ

Resource	Minimum	Recommended
OS	Ubuntu 20.04+ / WSL2 / macOS	Ubuntu 22.04
RAM	8 GB	16 GB+
Disk	5 GB	10 GB
Compiler	gcc/g++ 9+	gcc 11
Build tools	make, autoconf, libtool, python3	+ clang-tidy, boost

7.3 การติดตั้งบน Ubuntu / WSL2

```
# 1. ติดตั้ง dependencies
sudo apt-get update
sudo apt-get install -y \
    build-essential gcc g++ \
    autoconf automake libtool \
    python3 python3-dev \
    libboost-all-dev \
    libtcl-dev tk-dev \
    libreadline-dev \
    zlib1g-dev \
    git cmake

# 2. Clone source
cd ~
git clone https://github.com/ferrandi/PandA-bambu.git
cd PandA-bambu

# 3. Build
make -j$(nproc)

# 4. ทดสอบ
./bambu --version
```


7.4 การติดตั้งผ่าน Docker (แนะนำสำหรับ class)

```
docker pull christopherzimmerman/bambu:latest
docker run -it --rm -v $(pwd):/work christopherzimmerman/bambu:latest
```

7.5 โครงสร้าง Output ของ Bambu

เมื่อรัน Bambu จะได้ไฟล์ดังนี้:

```
out/
├─ HLS_output/
│  └─ verilog/
│     ├── dut.v                # top-level module
│     ├── dut_datapath.v       # datapath
│     ├── dut_controller.v     # FSM controller
│     ├── dut_slow_RAM_*       # inferred memory
│     └─ dut_tanhfxp_sigmoid_table.dat # LUT
│  └─ simulation/
│     ├── test_dut.v           # Verilog testbench
│     ├── test_dut.log         # simulation log
│     └─ results.txt           # PASS/FAIL
│  └─ HLS_summary.json
│  └─ HLS_synthesis_report.txt # resource report
└─ bambu_log.txt
```

7.6 คำสั่งพื้นฐาน

```
# แปลง C → Verilog (ไม่ simulate)
bambu \
  --top-fname=dut \
  file.c

# ระบุ target clock
bambu --top-fname=dut --clock-period=10 file.c

# Run co-simulation (C testbench + Verilog via Icarus)
bambu --top-fname=dut --simulate --simulator=VERILATOR file.c

# Generate for OpenROAD backend
bambu --top-fname=dut --backend=OpenROAD file.c

# ดู resource report
cat out/HLS_output/HLS_synthesis_report.txt
```

บทที่ 8 — Lab 1: Simple Adder

เป้าหมาย: เรียนรู้ Bambu HLS เบื้องต้น — ดูว่าโค้ด C ธรรมดากลายเป็น Verilog อย่างไร

8.1 ไฟล์ที่ต้องสร้าง

```
lab1-adder/  
├─ adder.c      # HLS source  
├─ adder.h      # header  
└─ run.sh       # script
```

8.2 `adder.c`

```
// adder.c  
#include "adder.h"  
  
void adder(int a, int b, int* c) {  
    *c = a + b;  
}  
  
int main() {  
    int a = 5, b = 7, c;  
    adder(a, b, &c);  
    if (c != 12) {  
        printf("FAIL: expected 12, got %d\n", c);  
        return 1;  
    }  
  
    a = -100; b = 200;  
    adder(a, b, &c);  
    if (c != 100) {  
        printf("FAIL: expected 100, got %d\n", c);  
        return 1;  
    }  
  
    printf("PASS\n");  
    return 0;  
}
```

8.3 `adder.h`

```
#ifndef ADDER_H
#define ADDER_H

#include <stdio.h>

void adder(int a, int b, int* c);

#endif
```

8.4 `run.sh`

```
#!/usr/bin/env bash
set -euo pipefail

LAB_DIR="$(cd "$(dirname "$0")" && pwd)"
cd "$LAB_DIR"

echo "====="
echo "Lab 1: Simple Adder"
echo "====="

# Step 1: HLS synthesis (C -> Verilog)
echo "[1/3] Running Bambu HLS..."
bambu \
  --top-fname=adder \
  -O2 \
  --clock-period=10 \
  --generate-tb=testbench=main \
  --simulate \
  --simulator=VERILATOR \
  --print-dot \
  adder.c

# Step 2: Inspect output
echo ""
echo "[2/3] Generated files:"
ls -la HLS_output/verilog/

# Step 3: View resource report
echo ""
echo "[3/3] Resource report:"
cat HLS_output/HLS_synthesis_report.txt
```

8.5 การรัน

```
cd lab1-adder
chmod +x run.sh
./run.sh
```

8.6 ผลลัพธ์ที่คาดหวัง

```
=====
Lab 1: Simple Adder
=====
[1/3] Running Bambu HLS...
===== Bambu Log =====
...
[SUCCESS] Co-simulation finished: PASSED
=====
[2/3] Generated files:
HLS_output/verilog/adder.v
HLS_output/verilog/adder_datapath.v
HLS_output/verilog/adder_controller.v
[3/3] Resource report:
=====
== Bambu HLS Resource Report
=====
Module: adder
Resource utilization:
  LUTs:    ~10
  FFs:     ~50
  DSPs:    0
  BRAMs:   0
  Latency: 1 cycle
  Interval: 1 cycle
```

8.7 Verilog ที่ได้ — ดูด้วยตา

```
cat HLS_output/verilog/adder.v
```

จะเห็น:

```

module adder(
    input  clock,
    input  reset,
    input  start_port,
    output done_port,
    input  [31:0] a,
    input  [31:0] b,
    output [31:0] c,
    // ... control signals
);
// FSM
reg [1:0] _tmp_1;
always @(posedge clock or posedge reset) begin
    if (reset) _tmp_1 <= 2'd0;
    else _tmp_1 <= /* next state */;
end
// datapath
assign c = a + b;
endmodule

```

8.8 แบบฝึกหัด

1. เปลี่ยน `int` เป็น `char` (8-bit) — ดู resource report เปลี่ยนอย่างไร
2. เพิ่ม `* 3` หลังการบวก — ดูว่าเกิด DSP ในรายงานหรือไม่
3. ลอง `--clock-period=5` (เร็วขึ้น) — Bambu อาจเพิ่ม pipeline stage

บทที่ 9 — Lab 2: FIR Filter

เป้าหมาย: เรียนรู้การใช้ `#pragma HLS PIPELINE` เพื่อเพิ่ม throughput

9.1 ทฤษฎี FIR

FIR (Finite Impulse Response) filter:

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k]$$

ใช้ shift register + MAC (multiply-accumulate)

9.2 fir.c

```

// fir.c
// 8-tap FIR filter, 16-bit signed input
#include "fir.h"

#define N_TAPS 8

// Filter coefficients (low-pass, normalized)
const int coeffs[N_TAPS] = {
    100, 250, 500, 700, 700, 500, 250, 100
};

void fir(int x, int* y) {
    static int shift_reg[N_TAPS] = {0};

#pragma HLS PIPELINE II=1
    // Shift register
    for (int i = N_TAPS - 1; i > 0; i--) {
        shift_reg[i] = shift_reg[i-1];
    }
    shift_reg[0] = x;

    // MAC
    int acc = 0;
    for (int k = 0; k < N_TAPS; k++) {
        acc += shift_reg[k] * coeffs[k];
    }
    *y = acc;
}

int main() {
    int x, y;

    // Test 1: impulse response
    // Input: 1, 0, 0, ..., 0 → output: coeffs[0], coeffs[1], ...
    x = 1024; fir(x, &y); // y should ≈ 100 * 1024
    if (y < 90000 || y > 110000) {
        printf("FAIL T1: got %d\n", y); return 1;
    }

    for (int i = 0; i < N_TAPS; i++) {
        x = 0; fir(x, &y);
    }
    // After N_TAPS zeros, y should be ~0

    // Test 2: step response
    for (int i = 0; i < 20; i++) {
        x = 1024; fir(x, &y);
    }
    // Should be ~ sum(coeffs) * 1024

```

```

    printf("PASS\n");
    return 0;
}

```

9.3 fir.h

```

#ifndef FIR_H
#define FIR_H

#include <stdio.h>

void fir(int x, int* y);

#endif

```

9.4 run.sh

```

#!/usr/bin/env bash
set -euo pipefail
cd "$(dirname "$0")"

echo "=== Lab 2: FIR Filter with Pipelining ==="

# Baseline (no directive)
echo ""
echo "--- A. Baseline (no PIPELINE) ---"
bambu --top-fname=fir --clock-period=10 --simulate fir.c \
    2>&1 | tail -20

# With PIPELINE II=1
echo ""
echo "--- B. With PIPELINE II=1 ---"
bambu --top-fname=fir --clock-period=10 --simulate fir.c \
    --pragma="HLS PIPELINE II=1" \
    2>&1 | tail -20

# Resource comparison
echo ""
echo "--- C. Resource Report ---"
cat HLS_output/HLS_synthesis_report.txt

```

9.5 ผลลัพธ์ที่คาดหวัง

Configuration	Latency	II	DSPs	LUTs
No PIPELINE	~50 cycles	50	1	~200
PIPELINE II=1	~10 cycles	1	8	~800

สังเกต: - PIPELINE II=1 → DSP เพิ่มขึ้น 8 (เพราะ unroll implicit) - Throughput เพิ่มขึ้น 50x - Area เพิ่มขึ้น 4x — trade-off!

9.6 แบบฝึกหัด

- 1. ลอง PIPELINE II=2 — area vs throughput เปลี่ยนอย่างไร
- 2. เพิ่ม N_TAPS เป็น 16, 32 — observe scaling
- 3. ใช้ ap_int<16> แทน int (Bambu ใช้ ac_int<16,true>) — observe DSP/LUT

บทที่ 10 — Lab 3: Matrix Multiplication

เป้าหมาย: เรียนรู้ ARRAY_PARTITION + UNROLL เพื่อจัดการ memory bottleneck

10.1 โค้ด

```
// matmul.c
#include "matmul.h"

#define N 8

void matmul(int A[N][N], int B[N][N], int C[N][N]) {
    #pragma HLS ARRAY_PARTITION variable=A complete dim=2
    #pragma HLS ARRAY_PARTITION variable=B complete dim=1
    #pragma HLS ARRAY_PARTITION variable=C complete dim=2

    for (int i = 0; i < N; i++) {
    #pragma HLS PIPELINE II=1
        for (int j = 0; j < N; j++) {
            int sum = 0;
            for (int k = 0; k < N; k++) {
    #pragma HLS UNROLL factor=4
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
}

int main() {
    static int A[N][N], B[N][N], C[N][N];

    // Initialize
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            A[i][j] = i + j;
            B[i][j] = i * 2 + j;
        }

    matmul(A, B, C);

    // Reference computation
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            int ref = 0;
            for (int k = 0; k < N; k++) {
                ref += A[i][k] * B[k][j];
            }
            if (C[i][j] != ref) {
                printf("FAIL at [%d][%d]: %d != %d\n", i, j, C[i][j], ref);
                return 1;
            }
        }
    }

    printf("PASS\n");
}
```

```

    return 0;
}

```

10.2 matmul.h

```

#ifndef MATMUL_H
#define MATMUL_H

#include <stdio.h>

#define N 8
void matmul(int A[N][N], int B[N][N], int C[N][N]);

#endif

```

10.3 run.sh

```

#!/usr/bin/env bash
set -euo pipefail
cd "$(dirname "$0")"
bambu --top-fname=matmul --clock-period=10 --simulate matmul.c
echo "--- Resource Report ---"
cat HLS_output/HLS_synthesis_report.txt

```

10.4 การวิเคราะห์

Optimization	Effect
ARRAY_PARTITION A dim=2	แยก row → เข้าถึง A[i][0..7] พร้อมกัน
ARRAY_PARTITION B dim=1	แยก column → เข้าถึง B[0..7][j] พร้อมกัน
UNROLL factor=4	4x parallel MAC
PIPELINE II=1	inner loop throughput สูงสุด

10.5 Trade-off Visualization

```

Throughput (II=1, full unroll):
→ 1 MAC / cycle
→ 64 cycles สำหรับ N=8

Latency (no optimization):
→ 8x8x8 = 512 cycles แบบ sequential

```

10.6 แบบฝึกหัด

1. เปลี่ยน N เป็น 4, 16 — observe synthesis time
 2. ลอง `BLOCK factor=4` แทน `complete` — compare resource
 3. ลอง `DATAFLOW` ระหว่าง for loop
-

บทที่ 11 — Lab 4: Dataflow Streaming

เป้าหมาย: เรียนรู้ task-level parallelism ด้วย DATAFLOW และ hls::stream

11.1 สถาปัตยกรรม

```
Input Stream → [Stage 1: +1] → [Stage 2: ×2] → [Stage 3: -5] → Output Stream
                FIFO1           FIFO2           FIFO3
```

11.2 dataflow.cpp

```
// dataflow.cpp
#include "dataflow.h"

void stage1(hls::stream<int>& in, hls::stream<int>& out) {
#pragma HLS INLINE off
    int x = in.read();
    out.write(x + 1);
}

void stage2(hls::stream<int>& in, hls::stream<int>& out) {
#pragma HLS INLINE off
    int x = in.read();
    out.write(x * 2);
}

void stage3(hls::stream<int>& in, hls::stream<int>& out) {
#pragma HLS INLINE off
    int x = in.read();
    out.write(x - 5);
}

void pipeline(hls::stream<int>& in, hls::stream<int>& out) {
#pragma HLS DATAFLOW
    hls::stream<int> mid1, mid2;
#pragma HLS STREAM variable=mid1 depth=4
#pragma HLS STREAM variable=mid2 depth=4

    stage1(in, mid1);
    stage2(mid1, mid2);
    stage3(mid2, out);
}

int main() {
    hls::stream<int> in, out;

    // Drive 10 samples
    for (int i = 0; i < 10; i++) {
        in.write(i);
    }

    // Run pipeline
    pipeline(in, out);

    // Check: out[i] should equal (i+1)*2 - 5
    for (int i = 0; i < 10; i++) {
        int got = out.read();
        int expected = (i + 1) * 2 - 5;
        if (got != expected) {
            printf("FAIL at i=%d: got %d, expected %d\n", i, got, expected);
        }
    }
}
```

```

        return 1;
    }
}
printf("PASS\n");
return 0;
}

```

11.3 dataflow.h

```

#ifndef DATAFLOW_H
#define DATAFLOW_H

#include <hls_stream.h>
#include <stdio.h>

void pipeline(hls::stream<int>& in, hls::stream<int>& out);

#endif

```

11.4 run.sh

```

#!/usr/bin/env bash
set -euo pipefail
cd "$(dirname "$0")"
bambu --top-fname=pipeline --clock-period=10 --simulate dataflow.cpp
cat HLS_output/HLS_synthesis_report.txt

```

11.5 Pipeline Behavior

```

Clock:      0   1   2   3   4   5   6   7
Stage1:  [a] [b] [c] [d] ...
Stage2:      [a] [b] [c] ...
Stage3:          [a] [b] ...
output:              [a] [b] ...

```

- Latency ของ data point แรก: 3 cycles (fill pipeline)
- Throughput: 1 sample/cycle (หลังจาก pipeline fill)

11.6 แบบฝึกหัด

1. เพิ่ม stage 4: saturate ที่ 100
2. เปลี่ยน FIFO depth เป็น 1, 8 — observe area
3. ใช้ `ap_axis<>` แทน `int` เพื่อสร้าง AXI-Stream interface

บทที่ 12 — Verification

12.1 ระดับชั้นของ Verification ใน HLS

ระดับ	สิ่งที่ตรวจสอบ	เครื่องมือ
1. C algorithmic	logic ของ algorithm	gcc run, gdb
2. C/RTL co-sim	functional ของ generated RTL	Bambu -simulate + Icarus/Verilator
3. RTL standalone	standalone Verilog	Icarus, Verilator
4. Gate-level (post-synth)	หลัง Yosys/OpenROAD	Icarus + SDF
5. On hardware	bitstream โหลดลง FPGA	Vivado, LiteX

12.2 Bambu Co-Simulation แบบละเอียด

```
# ใช้ Verilator (เร็วกว่า Icarus)
bambu --top-fname=dut --simulate --simulator=VERILATOR file.c

# ใช้ Icarus
bambu --top-fname=dut --simulate --simulator=ICARUS file.c

# ใช้ ModelSim (commercial)
bambu --top-fname=dut --simulate --simulator=MODELSIM file.c
```

12.3 ดู Generated Testbench

```
cat HLS_output/simulation/test_dut.v
```

ไฟล์นี้ถูก generate อัตโนมัติ — คุณไม่ต้องเขียน HDL testbench

12.4 ใช้ COCOTB สำหรับ Advanced Verification

COCOTB (Python-based testbench) ใช้กับ Verilog/VHDL ได้ — เหมาะกับ complex test:

```
cocotb_test.py
```

```

import cocotb
from cocotb.triggers import RisingEdge, Timer
from cocotb.clock import Clock

@cocotb.test()
async def test_fir(dut):
    # Clock 100 MHz
    cocotb.start_soon(Clock(dut.clock, 10, units="ns").start())

    # Reset
    dut.reset.value = 1
    dut.start_port.value = 0
    await RisingEdge(dut.clock)
    await RisingEdge(dut.clock)
    dut.reset.value = 0
    await RisingEdge(dut.clock)

    # Send 10 samples
    expected = [100, 350, 850, 1550, 2250, 2950, 3450, 3450, 2950, 2450]
    for i, exp in enumerate(expected):
        dut.a.value = 1024 if i == 0 else 0
        dut.b.value = 0 # for simpler test
        dut.start_port.value = 1
        await RisingEdge(dut.clock)
        dut.start_port.value = 0
        # Wait done
        for _ in range(20):
            await RisingEdge(dut.clock)
            if dut.done_port.value == 1:
                break
        cocotb.log.info(f"sample {i}: y={int(dut.y.value)}")

```

Run:

```

cd HLS_output/verilog
cocotb-run --make

```

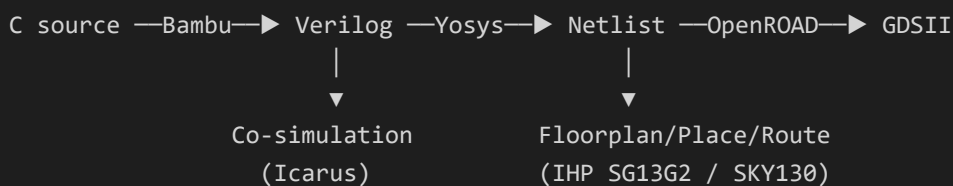
12.5 Self-Checking Testbench ใน C

```
int main() {
    int errors = 0;
    for (int t = 0; t < 100; t++) {
        int a = rand() % 1000;
        int b = rand() % 1000;
        int c;
        dut(a, b, &c);
        if (c != a + b) {
            printf("FAIL: %d + %d = %d (got %d)\n", a, b, a+b, c);
            errors++;
        }
    }
    return errors ? 1 : 0; // Bambu uses return value as PASS/FAIL
}
```

เคล็ดลับ: ใช้ return value 0 = PASS, ไม่ใช่ 0 = FAIL เพื่อให้ Bambu self-check ได้

บทที่ 13 — Integration กับ OpenROAD

13.1 Flow ภาพรวม



13.2 Step 1: Generate Verilog ด้วย Bambu

```
bambu \
  --top-fname=matmul \
  --clock-period=10 \
  --backend=OpenROAD \
  --memory-allocation-strategy=BAMBU \
  matmul.c
```

จะได้ไฟล์: - `HLS_output/verilog/matmul.v` — top module - `HLS_output/verilog/matmul_datapath.v` —
datapath - `HLS_output/verilog/matmul_controller.v` — FSM -
`HLS_output/verilog/matmul_slow_RAM_*.v` — inferred BRAMs

13.3 Step 2: Synthesize ด้วย Yosys

สร้าง `synth.py` :

```
# synth.py
read_verilog HLS_output/verilog/matmul.v
read_verilog -sv HLS_output/verilog/matmul_datapath.v
read_verilog -sv HLS_output/verilog/matmul_controller.v
read_verilog -sv HLS_output/verilog/matmul_slow_RAM_*.v

hierarchy -check -top matmul

# PDK-specific liberty file
read_liberty -lib /path/to/ihp-sg13g2_stdcell.lib

synth -top matmul
abc -liberty /path/to/ihp-sg13g2_stdcell.lib
write_verilog synth/matmul_synth.v
write_json synth/matmul.json
```

Run:

```
yosys -s synth.py
```

13.4 Step 3: OpenROAD Flow

`run_openroad.tcl` :

```

# run_openroad.tcl
set design matmul
set top_module matmul
set pdk_dir /path/to/IHP-Open-PDK

# Read Liberty
read_liberty ${pdk_dir}/libs.ref/sg13g2_stdcell/lib/sg13g2_stdcell.lib

# Read netlist
read_verilog synth/matmul_synth.v
link_design $top_module

# Constraints
create_clock -name clk -period 10 [get_ports clk]
set_input_delay 2 -clock clk [all_inputs]
set_output_delay 2 -clock clk [all_outputs]

# Floorplan
initialize_floorplan \
    -die_area "0 0 200 200" \
    -core_area "10 10 190 190" \
    -site unithv

# Place
global_placement -density 0.6
detailed_placement

# CTS
clock_tree_synthesis -root_buf sg13g2_buf_8

# Route
global_routing
detailed_routing

# Reports
report_timing > reports/timing.rpt
report_area > reports/area.rpt
write_def synth/matmul.def
write_gdsii synth/matmul.gds

```

13.5 LibreLane Flow (อัตโนมัติ)

สร้าง `config.json` :

```
{
  "DESIGN_NAME": "matmul",
  "VERILOG_FILES": [
    "HLS_output/verilog/matmul.v",
    "HLS_output/verilog/matmul_datapath.v",
    "HLS_output/verilog/matmul_controller.v",
    "HLS_output/verilog/matmul_slow_RAM_*.v"
  ],
  "CLOCK_PERIOD": 10,
  "CLOCK_PORT": "clock",
  "PDK": "ihp-sg13g2",
  "FP_CORE_UTIL": 50,
  "FP_SIZING": "absolute",
  "DIE_AREA": [0, 0, 200, 200],
  "CORE_AREA": [10, 10, 190, 190]
}
```

```
librelane --config config.json --tag hls-run .
```

13.6 SKY130 Variant

```
{
  "DESIGN_NAME": "matmul",
  "VERILOG_FILES": [...],
  "CLOCK_PERIOD": 10,
  "PDK": "sky130A",
  "STD_CELL_LIBRARY": "sky130_fd_sc_hd"
}
```

13.7 ผลลัพธ์ที่คาดหวัง (Post-Route)

Metric	Lab 1 (Adder)	Lab 2 (FIR)	Lab 3 (MatMul)	Lab 4 (Dataflow)
Area (μm^2) @ IHP SG13G2	~500	~2,000	~8,000	~1,200
Fmax (MHz)	200	150	100	180
Latency (cycles)	1	~10	~80	~5
Throughput	1/cyc	1/cyc	1/cyc	1/cyc
Power (μW)	50	500	5,000	200

บทที่ 14 — Best Practices และ Pitfalls

14.1 Top 10 Pitfalls

1. Loop bound ไม่คงที่

```
// ❌ Bad
for (int i = 0; i < N; i++) // N จาก input
// ✅ Good (ใช้ #pragma)
#pragma HLS LOOP_TRIPCOUNT min=8 max=64
for (int i = 0; i < N; i++)
```

2. Pointer aliasing

```
// ❌ HLS assume alias
void foo(int* a, int* b) { *a = *b + 1; }
// ✅ ใช้ restrict
void foo(int* restrict a, int* restrict b) { *a = *b + 1; }
```

3. Array เข้าถึง conflict

```
// ❌ ต้อง read แล้ว write ที่เดียวกัน → conflict
buf[i] = buf[i-1] + 1;
// ✅ ใช้ shift register หรือ partition
```

4. Resource หหมด (II ไม่ลดลง)

Warning: Cannot schedule with II=1, minimum II = 4

→ เพิ่ม resource (UNROLL, partition) หรือ ยอมรับ II ที่สูงกว่า

5. Function ไม่ถูก inline

- เพิ่ม `INLINE` directive
- หรือ เปลี่ยนเป็น macro

6. Memory bottleneck

- ใช้ `ARRAY_PARTITION` หรือ local variable
- ใช้ streaming interface

7. Floating-point vs Fixed-point

- float ใช้ resource มาก → ใช้ fixed-point (`ac_fixed` , `ap_fixed`)
- tradeoff: range/precision

8. Static vs Global

- `static` ใน function = เก็บ state (เช่น shift_reg) → OK
- `static` นอก function = global → ระงับ initialization

9. Printf ใน synthesized function

- Bambu/Vitis จะ error
- เอา printf ออกจาก top function

10. ไม่ self-checking testbench

- return 0 = PASS, !=0 = FAIL
- Bambu จะใช้ตรวจ co-sim

14.2 Debugging Workflow

1. Compile C with gcc → ตรวจ algorithm ก่อน
2. Run Bambu --simulate → ตรวจ RTL functional
3. ดู HLS_synthesis_report.txt → ตรวจ resource
4. ดู generated Verilog → ตรวจ micro-architecture
5. ใช้ waveform (--generate-vcd) → debug cycle-by-cycle

Generate waveform:

```
bambu --top-fname=dut --simulate --generate-vcd --vcd-fname=dut.vcd dut.c
gtkwave dut.vcd
```

14.3 Performance Tuning Recipe

```
// เริ่มจาก baseline
void kernel(...) { ... }

// Level 1: PIPELINE บน inner loop
#pragma HLS PIPELINE II=1
for (...) ...

// Level 2: ARRAY_PARTITION ถ้า memory bottleneck
#pragma HLS ARRAY_PARTITION variable=arr complete

// Level 3: UNROLL บน inner-most
#pragma HLS UNROLL factor=N

// Level 4: DATAFLOW ระหว่าง function
#pragma HLS DATAFLOW

// Level 5: เปลี่ยน algorithm (loop interchange, tiling)
```

14.4 การเลือก Optimization Level

Goal	Strategy
Minimize area	ไม่ PIPELINE, share resource, เล็ก type
Maximize throughput	PIPELINE II=1 + UNROLL + PARTITION
Balance	PIPELINE II=2-4, partial UNROLL
Minimize latency	DATAFLOW + parallel stages

A.1 Bambu Command-Line Quick Reference

```
# Basic
bambu --top-fname=fn file.c

# Common flags
--top-fname=NAME           # top function name
--clock-period=NS         # target clock period (ns)
-00, -01, -02, -03       # optimization level
--device=xc7a100tcsg324-1 # target FPGA
--backend=OpenROAD        # ASIC backend

# Directives
--pragma="HLS PIPELINE II=1"
--pragma="HLS UNROLL factor=4"
--pragma="HLS ARRAY_PARTITION variable=arr complete"

# Simulation
--simulate                 # run C/RTL co-simulation
--simulator=VERILATOR     # verilator
--simulator=ICARUS        # icarus
--generate-vcd             # output waveform
--vcd-fname=trace.vcd

# Output
--output-dir=out          # output directory
--print-dot               # print CDFG as .dot

# Memory
--memory-allocation-strategy=BAMBU
--bram-load-bytes=4
```

A.2 Vitis HLS ↔ Bambu Directive Mapping

Concept	Vitis HLS	Bambu
Pipeline loop	<code>#pragma HLS PIPELINE</code>	เหมือนกัน
Unroll	<code>#pragma HLS UNROLL</code>	เหมือนกัน
Partition	<code>#pragma HLS ARRAY_PARTITION</code>	เหมือนกัน
Dataflow	<code>#pragma HLS DATAFLOW</code>	เหมือนกัน
Interface	<code>#pragma HLS INTERFACE</code>	Bambu auto-infer
Arbitrary int	<code>ap_int<N></code>	<code>ac_int<N,true/false></code>
Stream	<code>hls::stream<T></code>	<code>hls::stream<T></code>
Fixed point	<code>ap_fixed<Q,N></code>	<code>ac_fixed<Q,N,...></code>

A.3 Common File Templates

Top function template:


```

#include "types.h" // ap_int / ac_int
#include "hls_stream.h" // hls::stream

void dut_top(
    // input data
    int in_data,
    // output
    int* out_data
) {
    // static for state
    static int state_reg = 0;

#pragma HLS PIPELINE II=1
    // compute
    *out_data = state_reg + in_data;
    state_reg = in_data;
}

int main() {
    int in, out;
    in = 5; dut_top(in, &out);
    if (out != 0) return 1; // initial state

    in = 10; dut_top(in, &out);
    if (out != 5) return 1;

    in = 20; dut_top(in, &out);
    if (out != 10) return 1;

    return 0; // PASS
}

```

Stream-based template:

```

#include "hls_stream.h"
void dut_stream(
    hls::stream<int>& in,
    hls::stream<int>& out
) {
#pragma HLS PIPELINE II=1
    int x = in.read();
    out.write(x * 2);
}

```

A.4 Bambu Exit Codes

Code	Meaning
0	Success (PASS)
1	Synthesis failed
2	Co-simulation mismatch (FAIL)
3	Timeout

A.5 Useful Bambu Log Sections

```
# Resource utilization
grep -A 20 "Resource utilization" out/HLS_output/HLS_synthesis_report.txt

# Latency
grep "Latency\|II " out/HLS_output/HLS_synthesis_report.txt

# Critical path
grep "Critical" out/HLS_output/HLS_synthesis_report.txt

# Errors
grep -i "error" out/bambu_log.txt
```

A.6 IHP SG13G2 + HLS เฉพาะ

Issue	Solution
HLS uses too much area	ลด UNROLL, ใช้ fixed-point
BRAM ไม่พอ	ใช้ distributed RAM, ลด array size
ไม่มี hardware multiplier ใน low-density	ใช้ shift-and-add (Bambu auto)
Clock ไม่ถึง target	เพิ่ม pipeline stage, สบ DATAFLOW

A.7 References

1. **Bambu Documentation:** https://panda.dei.polimi.it/?page_id=31
2. **PandA Framework:** <https://github.com/ferrandi/PandA-bambu>
3. **Xilinx UG902** (Vitis HLS User Guide): <https://docs.xilinx.com>
4. **OpenROAD Flow:** <https://openroad.readthedocs.io>
5. **IHP SG13G2 PDK:** <https://github.com/IHP-GmbH/IHP-Open-PDK>

6. SKY130 PDK: <https://skywater-pdk.readthedocs.io>

7. COCOTB: <https://docs.cocotb.org>

8. “High-Level Synthesis: From Algorithm to Digital Circuit” — Philippe Coussy et al., Springer 2008

แบบฝึกหัดรวม (Final Project)

โจทย์: ออกแบบ **Sobel Edge Detector** ด้วย HLS

1. Input: 64×64 grayscale image (8-bit pixel)

2. Output: 64×64 edge-magnitude image

3. Filter kernel:

- $G_x = \begin{bmatrix} [-1, 0, 1], [-2, 0, 2], [-1, 0, 1] \end{bmatrix}$
- $G_y = \begin{bmatrix} [-1, -2, -1], [0, 0, 0], [1, 2, 1] \end{bmatrix}$
- $\text{magnitude} = \sqrt{G_x^2 + G_y^2}$ (หรือ approximation $|G_x| + |G_y|$)

งานที่ต้องส่ง: 1. Source code C/C++ (`sobel.c`) 2. Bambu HLS report 3. Generated Verilog 4. Resource utilization (LUT, FF, DSP, BRAM) 5.

(Bonus) นำไปสังเคราะห์ผ่าน OpenROAD ด้วย IHP SG13G2 หรือ SKY130

Hint: - ใช้ `static` array สำหรับ line buffer - ใช้ PIPELINE + ARRAY_PARTITION - ทดลองทั้ง 2 precision: int vs fixed-point - เปรียบเทียบ resource ก่อน/หลัง optimization

จบหลักสูตร

หากมีคำถามเพิ่มเติมหรือต้องการให้ทำ slide PPTX, สร้างเพิ่ม Lab เฉพาะทาง (เช่น HLS สำหรับ ML accelerator, RISC-V pipeline), หรือต้องการเวอร์ชัน PDF/PPTX — แจ้งได้เลยครับ

- Introduction to High-Level Synthesis — Training Course
 - โครงสร้างหลักสูตร
 - Labs
 - Quick Start
 - Learning Path
 - เนื้อหาในแต่ละไฟล์
 - hls-course-part1-theory.md
 - hls-course-part2-practice.md
 - Prerequisites สำหรับนักศึกษา
 - เอกสารอ้างอิง
 - License

Introduction to High-Level Synthesis — Training Course

ระดับ: Beginner → Intermediate ระยะเวลา: 3 วัน (ทฤษฎี 1 วัน + ปฏิบัติ 2 วัน) เครื่องมือ: Bambu HLS + Yosys + OpenROAD + Icarus + COCOTB PDK: IHP SG13G2 (130 nm SiGe BiCMOS) และ SKY130

โครงสร้างหลักสูตร

ส่วน	ไฟล์	เนื้อหา
Part I — ทฤษฎี	<code>hls-course-part1-theory.md</code>	บทที่ 1–6: แนวคิด HLS, Flow, Scheduling/Binding, Coding Style, Directives, Interfaces
Part II — ปฏิบัติ	<code>hls-course-part2-practice.md</code>	บทที่ 7–14: ติดตั้ง Bambu, Labs 1–4, Verification, OpenROAD integration, Best Practices
Labs	<code>labs/</code>	โค้ดตัวอย่างพร้อมรัน

Labs

Lab	โฟลเดอร์	หัวข้อ	Directives ที่ใช้
1	<code>labs/lab1-adder/</code>	Simple Adder (Hello World)	—
2	<code>labs/lab2-fir/</code>	8-tap FIR Filter	<code>PIPELINE II=1</code> + <code>UNROLL</code>
3	<code>labs/lab3-matmul/</code>	8x8 Matrix Multiplication	<code>ARRAY_PARTITION</code> + <code>UNROLL</code> + <code>PIPELINE</code>
4	<code>labs/lab4-dataflow/</code>	Streaming Pipeline	<code>DATAFLOW</code> + <code>hls::stream</code>

Quick Start

```
# 1. ติดตั้ง Bambu (ดูรายละเอียดใน Part II)
sudo apt-get install -y build-essential autoconf libtool libboost-all-dev libtcl-dev
tk-dev
git clone https://github.com/ferrandi/PandA-bambu.git
cd PandA-bambu && make -j$(nproc)

# 2. รัน Lab 1
cd hls-course/labs/lab1-adder
chmod +x run.sh
./run.sh
```

Learning Path

```
Day 1: ทฤษฎี
├─ Morning: บทที่ 1-3 (แนวคิด, Flow, Internal ops)
├─ Afternoon: บทที่ 4-6 (Coding Style, Directives, Interfaces)
└─ Setup: ติดตั้ง Bambu + Yosys + OpenROAD

Day 2: ปฏิบัติ
├─ Morning: Lab 1 (Adder), Lab 2 (FIR)
├─ Afternoon: Lab 3 (MatMul)
└─ Homework: เปลี่ยน optimization, observe resource

Day 3: ปฏิบัติ (ขั้นสูง)
├─ Morning: Lab 4 (Dataflow), Verification (COCOTB)
├─ Afternoon: OpenROAD integration
└─ Final Project: Sobel Edge Detector
```

เนื้อหาในแต่ละไฟล์

[hls-course-part1-theory.md](#)

- บทที่ 1: HLS คืออะไร, ประวัติ, เครื่องมือ
- บทที่ 2: Compilation Flow (Frontend → Transform → Schedule → Bind → RTL)
- บทที่ 3: CDFG, Scheduling algorithms, Resource allocation
- บทที่ 4: Arbitrary precision types, Coding style
- บทที่ 5: PIPELINE, UNROLL, ARRAY_PARTITION, DATAFLOW
- บทที่ 6: Block-level interface, AXI, Stream, BRAM

hls-course-part2-practice.md

- บทที่ 7: ติดตั้ง Bambu, โครงสร้าง output
- บทที่ 8: Lab 1 - Simple Adder
- บทที่ 9: Lab 2 - FIR Filter
- บทที่ 10: Lab 3 - Matrix Multiplication
- บทที่ 11: Lab 4 - Dataflow Streaming
- บทที่ 12: Verification ด้วย Icarus + COCOTB
- บทที่ 13: OpenROAD integration
- บทที่ 14: Best practices + Cheat-sheet

Prerequisites สำหรับนักศึกษา

- พื้นฐาน Verilog/SystemVerilog
- เคยใช้ Icarus Verilog หรือ Vivado
- พื้นฐาน C/C++ (pointers, arrays, structs)
- Linux command line

เอกสารอ้างอิง

1. Bambu HLS: https://panda.dei.polimi.it/?page_id=31
2. Xilinx UG902 (Vitis HLS): <https://docs.xilinx.com>
3. “High-Level Synthesis: From Algorithm to Digital Circuit” — Coussy et al.
4. OpenROAD: <https://openroad.readthedocs.io>
5. IHP-Open-PDK: <https://github.com/IHP-GmbH/IHP-Open-PDK>

License

เอกสารนี้ใช้สำหรับการศึกษา — ใช้และแชร์ได้อย่างอิสระ